

ADR-0002: Software Transactional Memory (STM) для управления конкурентным состоянием

Оперативная группа проекта SMP

Июль 2026

Статус

ACCEPTED (Принято к исполнению)

Контекст

Нашему L7-балансировщику `stm-markov-proxy` жизненно необходимо вести учет времени и распределять входящие сетевые запросы по пулу бэкендов в циклическом режиме (Round-Robin). Это означает, что тысячи независимых грин-потокков будут одновременно ломиться к одной и той же переменной-счетчику, пытаться прочитать текущий индекс, увеличить его на единицу и записать обратно.

Если использовать классические блокировки из мира Unix (мьютексы или семафоры), мы неизбежно столкнемся с двумя проблемами. Во-первых, блокировки не композируются — их тяжело объединять в сложные цепочки без риска намертво заклинить систему (Deadlock). Во-вторых, потоки будут тратить процессорные такты на ожидания в очереди за «замком», что убьет производительность под обстрелом утилиты стресс-тестирования `wrk`.

Решение

Вместо пессимистичных блокировок утверждается использование **Software Transactional Memory (STM)**. Вся работа с динамическим счетчиком переносится в стерильную рантайм-лабораторию GHC:

1. Глобальное изменяемое состояние заворачивается в транзакционную переменную `TVar ProxyState` и помещается внутрь общего регистра `Env`.

2. Любое изменение счетчика происходит внутри изолированной монады STM через механизм TLog (Transaction Log). Поток делает копию данных в свой локальный «черновик» и крутит логику там, не выставляя никаких замков физической памяти и никому не мешая.
3. Мостом в реальный мир эффектов ввода-вывода становится атомарный комбинатор:

$$\text{atomically} :: STM\ a \rightarrow IO\ a$$

Рантайм Haskell проверяет черновик на финише через процессорную инструкцию CAS (Compare-And-Swap). Если за время вычислений другой поток успел обновить счетчик раньше, наш черновик тихо уничтожается (Rollback), и вся транзакция детерминированно запускается заново с первой строки (Retry), гарантируя кристальную чистоту внешнего мира.

Последствия

- **Плюсы:** Полная математическая защита от дедлоков под высокой нагрузкой. Жесткая изоляция эффектов на уровне системы типов GHC — компилятор физически не пропустит сетевой или консольный IO-код внутрь транзакции STM. Высокая скорость за счет работы с персистентными указателями без глубокого копирования объектов в памяти.
- **Минусы:** Риск высокой частоты перезапусков транзакций (Transaction Contention) при экстремальном конкурентном доступе на запись к одной переменной, что требует обязательного аудита под нагрузкой.